

Task Assignment With Efficient Federated Preference Learning in Spatial Crowdsourcing

Hao Miao , Xiaolong Zhong , Jiaxin Liu , Yan Zhao , Xiangyu Zhao , Weizhu Qian ,
Kai Zheng , *Senior Member, IEEE*, and Christian S. Jensen , *Fellow, IEEE*

Abstract—Spatial Crowdsourcing (SC) is finding widespread application in today's online world. As we have transitioned from desktop crowdsourcing applications (e.g., Wikipedia) to SC applications (e.g., Uber), there is a sense that SC systems must not only provide effective task assignment but also need to ensure privacy. To achieve these often-conflicting objectives, we propose a framework, Task Assignment with Federated Preference Learning, that performs task assignment based on worker preferences while keeping the data decentralized and private in each platform center (e.g., each delivery center of an SC company). The framework includes a federated preference learning phase and a task assignment phase. Specifically, in the first phase, we build a local preference model for each platform center based on historical data. We provide means of horizontal federated learning that makes it possible to collaboratively train these local preference models under the orchestration of a central server. Specifically, we provide a practical method that accelerates federated preference learning based on stochastic controlled averaging and achieves low communication costs while considering data heterogeneity among clients. The task assignment phase aims to achieve effective and efficient task assignment by considering workers' preferences. Extensive evaluations on real data offer insight into the effectiveness and efficiency of the paper's proposals.

Index Terms—Preference, task assignment, federated learning, spatial crowdsourcing.

I. INTRODUCTION

THE last decade has witnessed substantial advances in Spatial Crowdsourcing (SC), which enables people on the

move to serve as multi-modal sensors that perform a variety of location-based tasks [1], [2], [3]. The rising concerns of privacy [4], [5], [6], [7], [8], [9], [10] and efficiency [11], [12], [13], [14], [15] raised by SC are continuously attracting attention. In particular, privacy is desirable in SC. In order to achieve effective SC services, workers or platform centers (e.g., delivery centers of an SC company) are usually required to disclose their raw information (e.g., workers' locations and historical data). However, it is dangerous that real data can be used by a malicious third party. Thus, people will be unwilling to hand over their data to an SC platform, which leads to low worker participation and even worker churn.

Previous studies on privacy protection in SC focus mainly on protecting the location information of workers or tasks [4], [6], [10], [16], [17], data federation [18], [19], and on secure distance computation [5], [7], [9]. However, these studies do not take into account worker preferences, which are essential for the operation of an SC platform. These are needed in order to assign workers appropriate tasks, which is key to ensuring continuous worker participation and satisfaction. In contrast, when a worker is assigned an inappropriate task, the worker may complete the task with low quality or may even sabotage the task, which impacts the SC platforms negatively. Considering worker preferences is thus crucial in SC. Recently, a number of studies have explored worker preference in SC [11], [13], [14], [15]. Zhao et al. [15] model different workers' preferences for different categories of tasks in different time slots with a three-dimensional tensor and fill-in missing entries in tensors based on workers' task-performance histories and context matrices. Another study [11] uses historical task-performance data to maximize the mutual information among workers in order to learn informative representation vectors of groups and further learn group preferences. These studies do not consider privacy when modeling worker preferences. Instead, they transmit raw data directly to the SC platform and train a preference model in a centralized manner. In practice, data transfer between platform centers is cumbersome, and platform centers may be unwilling to share their data with other centers. In this kind of setting, task assignment often fails to achieve effective task assignment. To address the above challenges, we provide a preference-driven task assignment solution that integrates federated learning to protect raw data while using workers' preferences in task assignment. Federated Learning (FL) is a machine learning approach where many clients (e.g., mobile devices, organizations, or platforms) collaboratively train a model under the orchestration

Manuscript received 26 February 2023; revised 8 July 2023; accepted 26 August 2023. Date of publication 5 September 2023; date of current version 8 March 2024. This work was supported in part by NSFC under Grants 61972069, 61836007, 61832017, and 62272086, in part by Shenzhen Municipal Science and Technology R&D Funding Basic Research Program under Grant JCYJ20210324133607021, in part by the Municipal Government of Quzhou under Grant 2022D037, and in part by the Key Laboratory of Data Intelligence and Cognitive Computing, Longhua District, Shenzhen. Recommended for acceptance by Y. Tong. (*Corresponding author: Yan Zhao.*)

Hao Miao, Yan Zhao, and Christian S. Jensen are with the Department of Computer Science, Aalborg University, 9220 Aalborg, Denmark (e-mail: haom@cs.aau.dk; yanz@cs.aau.dk; csj@cs.aau.dk).

Xiaolong Zhong and Jiaxin Liu are with the School of Computer Science and Engineering, University of Electronic Science and Technology of China, Chengdu 610054, China (e-mail: xiaolongzhong2000@std.uestc.edu.cn; jiaxinliu1999@std.uestc.edu.cn).

Xiangyu Zhao is with the School of Data Science, City University of Hong Kong, Hong Kong, China (e-mail: xianzhao@cityu.edu.hk).

Weizhu Qian is with the School of Computer Science and Technology, Soochow University, Suzhou 215006, China (e-mail: wqian@cs.aau.dk).

Kai Zheng is with the Yangtze Delta Region Institute (Quzhou), School of Computer Science and Engineering, and Shenzhen Institute for Advanced Study, University of Electronic Science and Technology of China, Chengdu 610054, China (e-mail: zhengkai@uestc.edu.cn).

Digital Object Identifier 10.1109/TKDE.2023.3311816

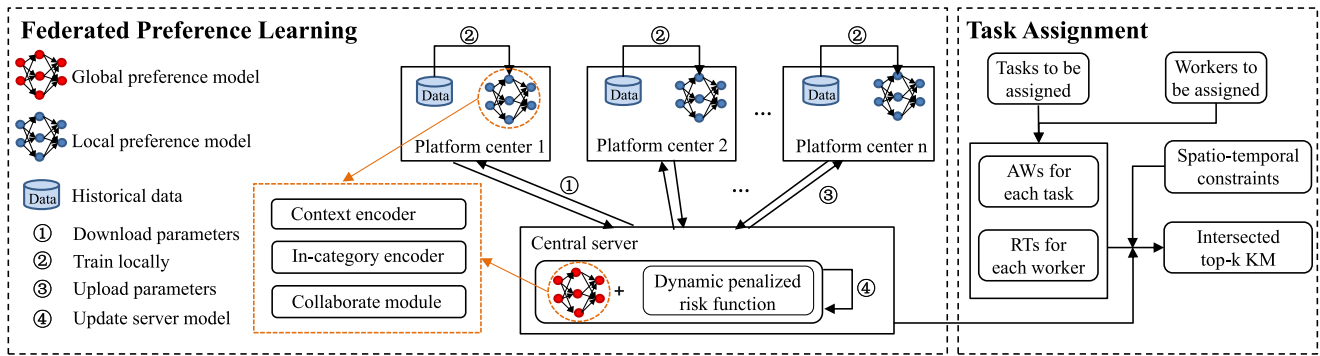


Fig. 1. Framework overview.

of a central server while keeping the training data decentralized. It embodies the principles of focused collection and data minimization, which can mitigate the systemic privacy risks and costs associated with traditional, centralized machine learning [20]. Moreover, FL addresses privacy concerns associated with the sharing of sensitive data with a central server, reducing the risk of data leakage. MaMahan et al. [21] show that federated learning offers many practical privacy benefits.

Filling the gap between existing research focusing on privacy issues and methods ignoring worker preferences, we propose a two-phase SC framework, namely Task Assignment with Federated Preference Learning (TA-FPL), which encompasses a federated preference learning and task assignment, as shown in Fig. 1. In the first phase, a federated preference model is built for each local center, and all local models are combined at a central server. For each preference model, each platform center first downloads parameters from the central server and utilizes local historical data to learn workers' preferences for all task categories by using a context encoder, an in-category encoder, and a collaboration module. Then, the central server receives the updated model parameters and aggregates them to update its global preference model. For the aggregation, we introduce a dynamic penalized risk function to achieve better performance. In the task assignment phase, we first calculate the Available Workers (AWs) for each task and the Reachable Tasks (RTs) for each worker based on spatio-temporal constraints. Then we propose an intersected top- k Kuhn Munkras (KM) algorithm, which considers the top- k AWs and RTs for each task and worker simultaneously, to achieve effective and efficient task assignment based on workers' preferences.

A preliminary version of this study [22] applies stochastic gradient descent (SGD) to optimize the federated preference learning, where clients calculate the gradients of the loss over their local data, and the central server then aggregates the gradients to update parameters. This approach requires a large number of training rounds to produce superior models, thus calling for a more communication-efficient optimization method. In real-world scenarios, the network connections between the server and clients can be unreliable due to many factors, and only a small subset of clients may be available at a given time. In addition, the distributions of the local data held by different clients are

often not independent and identically distributed (Non-IID) [23], which implies large heterogeneity.

To address these issues, we first optimized the training of preference learning in TA-FPL with Federated Averaging (FedAvg) [21]. Specifically, we select a C -fraction of the clients in each training round and update the selected clients locally multiple times by involving a server that performs model averaging. However, platform centers are generally heterogeneous (non-IID) and complex across different real-world SC settings. For instance, workers in one platform center may need to identify an attractive location for advertisements, while workers in another platform center may perform real-time traffic reporting. Such heterogeneous cases impact FedAvg negatively [23], [24], causing slow and unstable convergence due to client drift when updating clients. Client drift has the effect that the optimums of the loss functions of clients vary from client to client and from that of the server. To contend with client drift, we incorporate the idea of stochastic controlled averaging into the federated preference learning phase of TA-FPL, which enables overcoming heterogeneity and enables much faster convergence. More concretely, we consider the difference between the update direction for the server model c and each client c_i as the estimate of the client drift, which is then used to correct the local updates during local preference model training. The major value-added extensions over our preliminary study [22] are three-fold.

- 1) We analyze in depth a limitation of our previous federated training method that fails to solve non-IID SC data across clients, thus improving the efficiency of federated preference learning.
- 2) We provide a drift-correct federated training (DCFT) method that optimizes the preference learning by stochastic controlled averaging while contending with SC data heterogeneity, thereby achieving efficient communication between clients and the server.
- 3) We report on extensive experiments on a real-world dataset, offering evidence of the efficiency of the paper's proposals, which reduces the convergence time of preference learning by up to 45% compared to that of a federated preference learning (FPL) model.

The remainder of the paper is organized as follows. Preliminary concepts, notation, and a problem statement are provided in Section II. Next, federated preference learning and intersected

TABLE I
SUMMARY OF NOTATION

Symbol	Definition	Symbol	Definition
pc	Platform center	$w.l$	Current location of w
$pc.l$	Location of pc	$w.pc$	Platform center of w
$pc.W$	A worker set of pc	$w.r$	Reachable radius of w
s	Spatial task	$w.speed$	Speed of worker w
$s.l$	Location of s	$w.S$	A set of historical tasks of w
$s.p$	Publication time of s	A	A spatial task assignment
$s.e$	Expiration time of s	$A.S$	Allocated task set of S
$s.c$	Category of s	$\mathbb{A}$	Task assignment set
w	Worker		

Top- k KM task allocation are covered in Section III. We report on an experimental study in Section IV. Section V surveys related work, and Section VI offers conclusions.

II. PRELIMINARIES AND PROBLEM DEFINITION

We proceed to present preliminaries and define the problem addressed. Table I lists the notations used throughout the paper.

Definition 1 (Platform Center): A platform center, denoted by $pc = (l, W)$, has a location $pc.l$ and a set of workers $pc.W$.

Definition 2 (Spatial Task): A spatial task, denoted by $s = (l, p, e, c)$, has a location $s.l$, a publication time $s.p$, an expiration time $s.e$, and a category $s.c$.

With SC, the query of a spatial task s can be answered only if a worker is physically located at that location $s.l$ and be completed only if a worker arrives at $s.l$ before its deadline $s.e$. Note that with the single task assignment mode [25], an SC server should allocate each spatial task to only one worker at a time.

Definition 3 (Worker): A worker, denoted by $w = (l, pc, r, speed, S)$, has a location $w.l$, a center $w.pc$ that the worker w works for, a reachable radius $w.r$, a movement speed $w.speed$, and a set of historical tasks $w.S$.

The reachable range of worker w is a circle with $w.l$ as the center and $w.r$ as the radius, within which w can accept assignments. In our work, a worker can handle only one task at a certain time instance and belongs to a single center, which is reasonable in practice.

Definition 4 (Spatial Task Assignment): Given a set of workers $W = \{w_1, w_2, \dots, w_{|W|}\}$, a set of tasks $S = \{s_1, s_2, \dots, s_{|S|}\}$, and a set of platform centers $PC = \{pc_1, pc_2, \dots, pc_{|PC|}\}$, we define A as the spatial task assignment which consists of a set of tuples of form (pc, w, s) , where a spatial task s is assigned to worker w who works for pc , satisfying all the workers' and tasks' spatial-temporal constraints.

Based on the above definitions, a formal problem definition is as follows.

Preference-Driven Task Assignment With Privacy Protection: Given a set of centers PC with private local data (i.e., workers' historical task records and workers' locations), a set of online workers W , and a set of tasks S at the current timestamp, our problem is to find an optimal task assignment A_{opt} that maximizes the total number of assigned tasks, i.e., $\forall A_i \in \mathbb{A} (|A_{opt}.S| \geq |A_i.S|)$, by considering workers' preferences and protecting the privacy of each platform center, where $\mathbb{A}$ denotes all possible assignments and $A_i.S$ denotes the tasks of task assignment A_i .

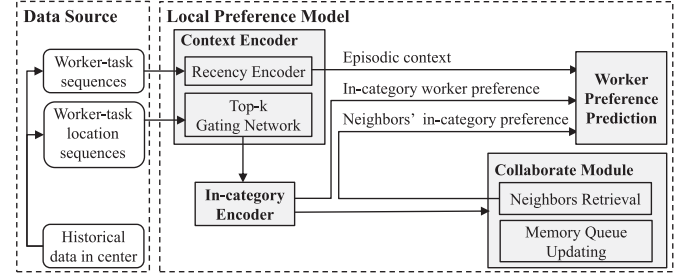


Fig. 2. Local preference model.

Note that the proposed TA-FPL encompasses a federated preference learning and a task assignment phase. Since we aim to learn workers' preferences in the federated learning phase, which also enables privacy, we do not define preference and privacy in the problem statement.

III. ALGORITHM

In this section, we present the proposed framework TA-FPL, which contains a federated preference learning phase, described in Section III-A, and a preference-driven task assignment phase, described in Section III-B. In the federated preference learning phase, a local preference model is proposed to model workers' preferences for local platform centers, which contains three modules, i.e., a context encoder, an in-category encoder, and a collaboration module. We combine these modules to predict the local workers' preferences. Then, we introduce a novel federated training process to update the parameters of the central server's model in order to get the global workers' preferences. In the preference-driven task assignment phase, we introduce an Intersected top- k KM algorithm to find a suitable task assignment.

A. Federated Preference Learning

In this section, we introduce the local preference model and federated model training, respectively. Noted that the central server's model and local platform centers' models share the same model framework, as shown in Fig. 2.

1) **Local Platform Center Preference Modeling:** Next, we explain how to use each platform center's local data to model workers' preferences. Given a set of workers W over a set of task locations L from a set of task categories C , a task record can be denoted as a tuple $s_i = (l_i, c_i)$, where i is the index of the task in a worker's historical task record sequence, l_i is the location of task s_i that the worker interacts with, and c_i is the task's category. Different tasks may be in the same location. A sequence of N tasks from worker w is denoted as $S_w = \{s_1, s_2, \dots, s_N\}$, which is ordered according to the chronological order of the tasks. $S_w^c = \{s_1^c, s_2^c, \dots, s_T^c\}$ represents the subsequence of S_w under category c , where T is the number of task records in this subsequence and task records in S_w^c are still in chronological order. Given S_w of worker w , the goal of our preference model is to predict this worker's task location and task category preferences in the near future.

As shown in Fig. 2, the local preference model is composed of three modules: a context encoder, an in-category encoder, and a collaboration module. First, to model worker preferences under a task category, a task sequence can be divided into multiple subsequences according to task categories, and each subsequence contains tasks of the same category. The in-category encoder, utilizing *SelfAttention* [26], [27], is used to model in-category transition patterns of task-to-task in the subsequences. Second, the context encoder contains episodic context and category context. Based on the categories of recent task records, the context encoder utilizes *SelfAttention* to predict the next category, thus helping determine a worker's preference for the next task location. To get the episodic context, the context encoder models the task-to-task transition patterns among recent task records with another *SelfAttention*. Third, due to the sparsity of observations in individual worker's task records, we retrieve workers with similar in-category preferences to the target worker based on the context encoder's next category prediction. Finally, task location and task category prediction are made based on the episodic context, the in-category worker preferences, and neighboring workers' in-category preferences. We next cover each component of this preference model.

Context Encoder: The context encoder is designed to obtain both the category and episodic context for worker preference prediction. To decide which in-category worker preferences should be used, the category of the next task is predicted. By using a top- k gating network and a recency encoder, we can obtain the category context and the episodic context in recent task records, respectively.

For the top- k gating network, we take the categories of recent tasks as input. Through the input category embedding layer, the categories of the most recent M task records can be projected into vectors $Z = [e_{c_{N-M}}^z, \dots, e_{c_N}^z]$, and the relative positions of recent task records $[M, \dots, 1]$ can be similarly projected into $P^z = [P_M^z, \dots, P_1^z]$ with the position embedding layer. Then a *SelfAttention* network [26], [27], composed of n_l layers of a multi-head attention block and a Fully Connected (FC) network block, transforms the category vectors $H^0 = Z + P^z$ into a hidden representations $H^{n_l} = [h_1^z, \dots, h_T^z]$. For the i th attention head, the input latent states H^j are transformed as shown in (1), where projection matrices W_i^Q, W_i^K , and W_i^V are learnable parameters and LN is a layer normalization network. We use $h^z = h_T^z$ to summarize the category information of recent task records.

$$A_i^j = \text{Attention} \left(H^j W_i^Q, H^j W_i^K, H^j W_i^V \right),$$

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{Q \cdot K^T}{\sqrt{d_a/n_h}} \right),$$

$$H^{j+1} = LN \left(H^j + FC^j(A^j) \right), \quad (1)$$

Next, we feed h^z into the output category embedding layer and a softmax layer. The top- k gating network generates a probability distribution over all task categories

$$p(\hat{c}_{N+1} = j) \propto \exp(\langle h^z, e_j^z \rangle), \quad (2)$$

where $p(\hat{c}_{N+1} = j)$ represents the probability of category j being the category of the next task location and e_j^z is the category embedding of category j . Considering the uncertainty in the prediction of the next category, the gating network selects the top- k most probable categories according to (3)

$$\{c_j\}_{j=1}^k = \underset{j'}{\text{argtopk}} \left(\{p(\hat{c}_{N+1} = j')\}_{j'=1}^{|C|} \right), \quad (3)$$

where $c_j \in C$.

The recency encoder is utilized to infer the episodic context from recent task records. Because of the continuity between tasks, the recent records of tasks reflect the ongoing intent of the next task location. For the recency encoder, we take the most recent M records of task locations $[l_{N-M}, \dots, l_N]$ in the original sequence S as the input, which is projected through the input task location embedding layer into vectors $X^r = [e_{l_{N-M}}, \dots, e_{l_N}]$. The relative positions of recent task records $[M, \dots, 1]$ are projected into $P^r = [P_M^r, \dots, P_1^r]$ through the position embedding layer similarly. Then, we use the *SelfAttention* network to transform vectors $X^r + P^r$ into hidden states, and the hidden state of the last task record is used to represent the inferred episodic context of h^r

$$h^r = \text{SelfAttention}(X^r + P^r). \quad (4)$$

The recency encoder checks the recent task locations beyond specific categories and provides a complementary view of the next task location besides the category context.

In-Category Encoder: With respect to the k predicted categories calculated according to (3), we can calculate the corresponding in-category worker preferences to predict the next task location, i.e., the hidden representations $\{h^{c_j}\}_{j=1}^k$ from the in-category encoder selected. Without loss of generality, we take the encoding process for a task subsequence of category c as an example. The corresponding task location subsequence $[l_1^c, \dots, l_T^c]$ is projected through the input task location embedding layer E_{in} into a set of dense vectors $X^c = [e_{l_1^c}, \dots, e_{l_T^c}]$. The relative positions $[T, \dots, 1]$ of these records to the next task location are projected through the position embedding layer P into $P^c = [P_T^c, \dots, P_1^c]$. Taking the dense vectors $X^c + P^c$ as input, the *SelfAttention* network outputs the representations of the worker preferences in this category h^c

$$h^c = \text{SelfAttention}(X^c + P^c). \quad (5)$$

Collaboration Module: By using collaborative learning among neighboring workers with similar preferences, the sparsity issue is mitigated. To predict the next task location, we combine the neighboring workers' information based on their similarities obtained with in-category subsequences. In the collaboration module, a memory tensor Mem is used to record workers' in-category preferences, storing the latent states of the last F workers for each category c in chronological order.

Using the target worker's in-category preferences h_w^c under category c , the reading operation of Mem can be performed as follows. First, we compute the similarity of the preferences under category c between workers w and i

$$\text{sim}(h_w^c, h_i^c) \propto \exp(\langle h_w^c, h_i^c \rangle). \quad (6)$$

Then we choose the *top-f* similar workers as the neighbors and take a weighted sum of their representations as the neighborhood representation for the next task location prediction

$$h^f = \sum_{i'}^{top-f} sim(h_w^c, h_{i'}^c) h_{i'}^c. \quad (7)$$

We initialize Mem for the writing operation: We then update it with the latest worker's in-category preference representations h^c , generated from the in-category encoder. Thus, the memory tensor is a queue that pushes the most recently served workers' representations of category c while dynamically popping out the representations of workers who have been inactive for a long time.

Preference Prediction: Based on the three modules above, we can get the worker preference of the task location and task category. We calculate the preference score of task category c_j with (2), $score(\hat{c}_j) = score(\hat{c}_{N+1} = c_j)$

$$score(\hat{c}_j) = exp(\langle h^z, e_j^z \rangle), \quad (8)$$

where h^z represents the category information of recent task records and e_j^z is the embedding of category j .

In order to predict the worker preference for task location l_i , $score(\hat{l}_i) = score(\hat{l}_{N+1} = l_i)$, we use the mixture of the obtained representations as the worker representation. We concatenate the representations of the in-category worker preferences of the predicted top- k categories h^{c_j} , the neighboring workers' in-category preferences h^{f_j} , and the episodic context inferred from the most recent task location records h^r , for each of the k categories and project the concatenated representations into the output task location embedding space with a fully connected (FC) network layer. Thus, the worker representation h_j for category j is

$$h_j = FC(h^r \oplus h^{c_j} \oplus h^{f_j}), \quad (9)$$

where $j = 1, 2, \dots, k$ and $\oplus$ is the concatenation operation. Then we use an inner product and softmax and get the worker preference for task location l_i under the premise of category j

$$score_j(\hat{l}_i) = softmax(\langle h_j, E_{out} \rangle), \quad (10)$$

where E_{out} represents the task locations' embeddings. Furthermore, when we consider all top- k categories, the worker's preference for task location l_i can be calculated according to (11)

$$score(\hat{l}_i) = \sum_{j=1}^k score_j(\hat{l}_i) score(\hat{c}_j). \quad (11)$$

2) Federated Preference Model Training: In this section, we introduce the global loss function and model parameters updating in a federated training way.

Loss Function: The loss function of each platform center has two parts: the loss of the task location prediction and the loss of the task category prediction.

For the loss of the task location prediction, we apply the negative sampling trick, which randomly samples N_s negative task locations according to their popularity in training datasets for

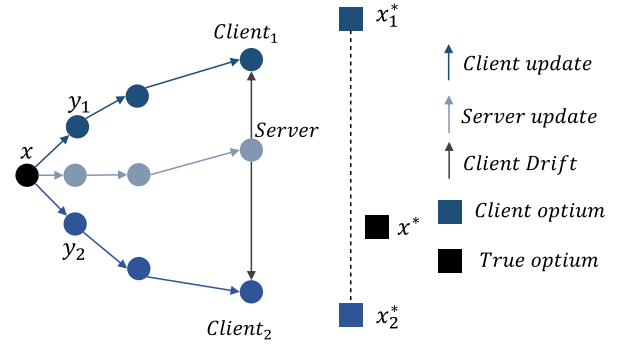


Fig. 3. Client-drift in FedAvg.

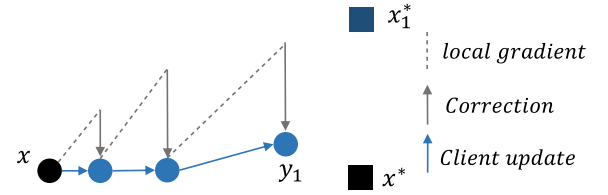


Fig. 4. Illustration of drift-correction.

each positive instances [28]. The loss of task location prediction is

$$L_{loc} = - \sum_{l=1}^{N_s+1} \delta(l_{N+1} = l) \log p(\hat{l}_{N+1} = l) \quad (12)$$

$$p(\hat{l}_{N+1} = l) = softmax(score(\hat{l}_{N+1} = l)), \quad (13)$$

where $\delta(\cdot)$ is the indicator function, l_{N+1} is the ground-truth task location, and $\hat{l}_{N+1}$ is the model's prediction. Likewise, we compute the loss of all task locations' categories

$$L_{cate} = - \sum_{j=1}^C \delta(c_{N+1} = j) \log p(\hat{c}_{N+1} = j), \quad (14)$$

where $p(\hat{c}_{N+1} = j)$ is computed by (2). We compute the joint loss as follows:

$$L = \lambda \times L_{loc} + (1 - \lambda) \times L_{cate}, \quad (15)$$

where λ is a hyper-parameter to control the weights.

Given a central server that can transmit and receive messages from m sampled platform centers, platform center pc_k consists of N_k training instances. The whole federated loss function $l(\theta)$ is

$$l(\theta) = \frac{1}{m} \sum_{k=1}^m L_k(\theta), \quad (16)$$

where $L_k(\theta)$ is the empirical loss (i.e., (15)) of platform center pc_k , and θ are the parameters of our preference network.

Federated Training: Initially, we employ federated training, as shown in Algorithm 1, to transmit local model parameters of each platform center to the central server for the purpose

Algorithm 1: Federated Preference Model Training.

Input: $T, \theta^0, \alpha, \theta_k^0$

- 1 **for** each round t in $[1, \dots, T]$ **do**
- 2 Sample platform centers P_t ;
- 3 Server transmits θ^{t-1} to each selected platform center;
- 4 **for** each platform center $pc_k \in P_t$ **do**
- 5 $\theta_k^t \leftarrow$
 $\arg \min_{\theta} L_k(\theta) - \langle \nabla L_k(\theta_k^{t-1}), \theta \rangle + \frac{\alpha}{2} \|\theta - \theta_k^{t-1}\|^2$;
- 6 $\nabla L_k(\theta_k^t) \leftarrow \nabla L_k(\theta_k^{t-1}) - \alpha(\theta_k^t - \theta_k^{t-1})$;
- 7 Transmit platform center model θ_k^t to central server;
- 8 **for** each platform center $pc_k \notin P_t$ **do**
- 9 $\theta_k^t \leftarrow \theta_k^{t-1}$;
- 10 $\nabla L_k(\theta_k^t) \leftarrow \nabla L_k(\theta_k^{t-1})$;
- 11 $h^t \leftarrow h^{t-1} - \frac{\alpha}{m} (\sum_{k \in P_t} \theta_k^t - \theta^{t-1})$;
- 12 $\theta^t \leftarrow (\frac{1}{|P_t|} \sum_{k \in P_t} \theta_k^t) - \frac{1}{\alpha} h^t$;

of privacy protection. In communication round $t \in [1, \dots, T]$, a subset of platform centers $P_t \subset \{pc_1, \dots, pc_N\}$ is active. The central server transmits its current model θ^{t-1} to these platform centers (lines 2–3). Each active platform center then optimizes a local empirical risk objective, which is the sum of its local empirical loss and a dynamic penalized risk function (line 19). In addition, each active platform center computes its local gradient to satisfy local optima condition (line 21) and then transmits the updated parameters to the central server (line 7). Unselected platform centers do not update their models (lines 8–10). Finally, the central server updates its state h^t , showing they converge to a point that turns out to be a stationary point of the global risk [29], and model parameters θ^t (lines 11–12).

However, previous studies [21] show that federated training requires many rounds to train a good model which incurs high communication overheads. To tackle this problem, we first train the preference learning with FedAvg, where a selected subset of clients first update their local models multiple times and where the server aggregates the average gradients from the selected clients.

FedAvg Based Training: We employ FedAvg to replace the federated training. As shown in Algorithm 2, we transmit local model parameters of selected platform centers to the central server by performing local training on these selected clients to reduce the computational load of the central server. Similarly, in a communication round $t \in [1, \dots, T]$, only a subset of all N platform centers $P_t \subset \{pc_1, \dots, pc_N\}$ are active, where N is the number of all platform centers. The central server allocates its current model θ^{t-1} to these platform centers (lines 2–3). But differently from federated training, each active platform center locally takes several steps of gradient descent on the current model using its local data with a local empirical risk objective, which contains a local empirical loss and a dynamic penalized risk function (line 6 and lines 11–19). Platform centers that were not selected do not update their models (lines 8–10). Finally, the central server updates its state h^t (which indicates whether they converge to a stationary point of the global risk [29]) and model parameters θ^t (lines 12–13).

Algorithm 2: FedAvg Based Preference Model Training.

Input: $T, \theta^0, \alpha, \theta_k^0, E$

- 1 **for** each round t in $[1, \dots, T]$ **do**
- 2 Sample $m(m \geq 1)$ platform centers P_t ;
- 3 Server transmits θ^{t-1} to each selected platform center;
- 4 **for** each client $k \in P_t$ in parallel **do**
- 5 $\theta_k^t \leftarrow \text{ClientUpdate}(k, \theta_k^{t-1})$;
- 6 **for** each platform center $pc_k \notin P_t$ **do**
- 7 $\theta_k^t \leftarrow \theta_k^{t-1}$;
- 8 $\nabla L_k(\theta_k^t) \leftarrow \nabla L_k(\theta_k^{t-1})$;
- 9 $h^t = h^{t-1} - \frac{\alpha}{m} (\sum_{k \in P_t} \theta_k^t - \theta^{t-1})$;
- 10 $\theta^t = (\frac{1}{|P_t|} \sum_{k \in P_t} \theta_k^t) - \frac{1}{\alpha} h^t$;
- 11 $\text{ClientUpdate}(k, \theta)$:
- 12 $\mathcal{B} \leftarrow$ split data stored in client k into batches of size B ;
- 13 **for** each local epoch i from 1 to E **do**
- 14 **for** batch $b \in \mathcal{B}$ **do**
- 15 $\theta_{k,i}^t \leftarrow \arg \min_{\theta} L_{k,i}(\theta) - \langle \nabla L_{k,i}(\theta_{k,i}^{t-1}), \theta \rangle +$
 $\frac{\alpha}{2} \|\theta - \theta_{k,i}^{t-1}\|^2$;
- 16 $\nabla L_{k,i}(\theta_{k,i}^t) \leftarrow \nabla L_{k,i}(\theta_{k,i}^{t-1}) - \alpha(\theta_{k,i}^t - \theta_{k,i}^{t-1})$;
- 17 $\theta_k^t \leftarrow \theta_{k,E}^t$;
- 18 Transmit platform center model θ_k^t to central server;
- 19 return θ_k^t .

Drift-Corrected Federated Training (DCFT): In some SC applications, the data held by different platform centers may vary considerably. Put differently, the data is heterogeneous and non-IID. For example, workers at one platform center may primarily deliver advertisements, while workers at another platform center may perform mostly real-time traffic reporting. Studies [23], [24] show that heterogeneity of SC data has adverse performance effects on FedAvg because it causes update drift when performing training on the clients. To illustrate client drift, Fig. 3 shows two clients with three update steps. For each client, the local updates move towards an individual optimum x_i^* , while the server updates move towards $\frac{1}{N} \sum_i x_i^*$ instead of the true optimum x^* , and $\frac{1}{N} \sum_i x_i^*$ may deviate substantially from x^* . Such situations may require more training rounds to achieve a good model, which degrades the performance of TA-FPL.

To contend with this issue, we propose so-called drift-corrected federated training that incorporates the idea of stochastically controlled averaging to optimize the preference learning. In other settings, this has proven to be able to enable stable training and faster convergence [24]. Specifically, we consider heterogeneity as ‘client-variance’ in the updates across different clients and aim to reduce the client-variance during the federated training. As illustrated in Fig. 4, when updating the model on a single client, the local gradient (dashed line) moves towards x_1^* , but we correct the update direction to ensure that the update points to the true optimum x^* by means of the correction term $(c - c_i)$, where c is server control factor (a state of the update direction) and c_i is the i -th client control factor. We initialize c and c_i to zero.

The training procedure of stochastic controlled averaging is similar to that of FedAvg, which has an additional drift

correction (i.e., $c - c_i$). Concretely, in communication round $t \in [1, \dots, T]$, we select a subset of platform centers $P_t \subset \{pc_1, \dots, pc_N\}$, and the server transmits its current model θ^{t-1} to these active platform centers (lines 3–4), as shown in Algorithm 2. Each active platform center updates locally the current model using its local data with a local empirical risk objective with drift correction (lines 15–25). Meanwhile, the local control variate c_i is also updated as follows,

$$c_i^+ = c_i - c + \frac{1}{|\mathcal{B}|\alpha} (\theta - \theta_k^t), \quad (17)$$

where the previously computed gradients are reused to update the control variate (line 22). Finally, the central server updates its state h_t , parameter θ^t , and control variate c to make the model move towards the optimum of the global model.

The difference between the original federated training and the optimized one (i.e., DCFT) is that DCFT considers data heterogeneity across platform centers (i.e., clients). This heterogeneity reduces preference learning because it causes update drift when performing training on the clients. DCFT features stochastically controlled averaging to optimize the preference learning, where heterogeneity is considered as ‘client-variance’ in the updates. Our aim is to reduce the client-variance during the federated training, as shown in lines 22 – 23 in Algorithm 3.

B. Preference-Driven Task Assignment

We proceed to first obtain the available worker set for each task and the reachable task set for each worker. Then, we use spatio-temporal and top- k constraints to build a Bipartite graph and propose an Intersected Top- k KM algorithm, which utilizes the worker preferences for different task categories calculated by the federated preference learning model. To protect each platform’s local data and to get the global workers’ preferences, all workers’ preferences are calculated in local platform centers and are then uploaded to the central server, where the task assignment occurs.

1) *Available Worker Set and Reachable Task Set*: Before building the Bipartite graph, we should consider spatio-temporal constraints to filter workers and tasks. Given a set of online workers, $W = \{w_1, w_2, \dots, w_{|W|}\}$ and a set of tasks, $S = \{s_1, s_2, \dots, s_{|S|}\}$, the available worker set for spatial task $s \in S$ and the reachable task set for worker $w \in W$ are denoted as $AW(s)$ and $RT(w)$, respectively.

Both $AW(s)$ ($\forall w \in AW(s), s \in S$) and $RT(w)$ ($\forall s \in RT(w), w \in W$) must satisfy two conditions: 1) $d(w.l, s.l) \leq w.r$, and 2) $t_{now} + d(w.l, s.l)/w.speed \leq s.e$, where $d(w.l, s.l)$ represents the distance between $w.l$ and $s.l$ (e.g., euclidean distance).

2) *Intersected Top- K KM Algorithm*: Here, we transform the task assignment problem into a Bipartite Maximum Weight Matching problem [30], which is based on a graph that is represented by $G = (V, E)$ with a set V of vertices and a set E of edges. Given a set of online workers, $W = \{w_1, w_2, \dots, w_{|W|}\}$, and a set of available tasks, $S = \{s_1, s_2, \dots, s_{|S|}\}$, the cardinality of V and the cardinality of E are fixed to $|W| + |S|$ and $\sum_{i=1}^{|W|} n_i$, respectively, where n_i is the number of worker w_i ’s

Algorithm 3: Drift-Corrected Federated Preference Model Training.

Input: $T, \theta^0, \alpha, \theta_k^0, E, c, c_i$

```

1 % Server execution
2 for each round  $t$  in  $[1, \dots, T]$  do
3   Sample  $m(m \geq 1)$  platform centers  $P_t$  from  $P$ ;
4   Server transmits  $\theta^{t-1}$  to each selected platform
   center;
5   for each client  $k \in P_t$  in parallel do
6      $\theta_k^t, c_i^+ \leftarrow ClientUpdate(k, \theta_k^{t-1})$ ;
7      $c_i = c_i^+$ 
8   for each platform center  $pc_k \notin P_t$  do
9      $\theta_k^t \leftarrow \theta_k^{t-1}$ ;
10     $\nabla L_k(\theta_k^t) \leftarrow \nabla L_k(\theta_k^{t-1})$ ;
11     $c \leftarrow c + \frac{1}{|P_t|} \sum_{i \in P_t} \delta_{c_i}$ 
12     $h^t \leftarrow h^{t-1} - \frac{\alpha}{m} (\sum_{k \in P_t} \theta_k^t - \theta^{t-1})$ ;
13     $\theta^t \leftarrow (\frac{1}{|P_t|} \sum_{k \in P_t} \theta_k^t) - \frac{1}{\alpha} h^t$ ;
14 % Update on client  $k$ 
15  $ClientUpdate(k, \theta)$ :
16    $\mathcal{B} \leftarrow$  split data stored in client  $k$  into batches of size
    $B$ ;
17   for each local epoch  $i$  from 1 to  $E$  do
18     for batch  $b \in \mathcal{B}$  do
19        $\theta_{k,i}^t \leftarrow \arg \min_{\theta} L_{k,i}(\theta) - \langle \nabla L_{k,i}(\theta_{k,i}^{t-1}), \theta \rangle +$ 
        $\frac{\alpha}{2} \|\theta - \theta^{t-1}\|^2$ ;
20        $\nabla L_{k,i}(\theta_{k,i}^t) \leftarrow \nabla L_{k,i}(\theta_{k,i}^{t-1}) - \alpha(\theta_{k,i}^t - \theta^{t-1} + c - c_i)$ ;
21      $\theta_k^t \leftarrow \theta_{k,E}^t$ ;
22      $c_i^+ \leftarrow c_i - c + \frac{1}{|\mathcal{B}|\alpha} (\theta - \theta_k^t)$ ;
23      $\delta_{c_i} \leftarrow c_i^+ - c_i$ ;
24     Transmit platform center model  $\theta_k^t, \delta_{c_i}$  to central
     server;
25   return  $\theta_k^t, c_i^+$ 
```

adaptive reachable assignments, which is a subset of worker w_i ’s reachable assignments $RT(w_i)$.

To construct vertices, V is partitioned into sets V_W and V_S . Each worker w_i maps to a vertex v_i^w , and each spatial task s_j maps to a vertex v_j^s . For the edges’ construction, we add an edge from v_i^w mapped from vertex $w_i \in W$ to vertex v_j^s mapped from $s_j \in S$ if they satisfy two conditions:

1) spatio-temporal constraints: $w_i \in AW(s_j)$ and $s_j \in RT(w_i)$, and

2) top- k constraints: $w_i \in \widetilde{AW}_k(s_j)$ and $s_j \in \widetilde{RT}_k(w_i)$, where $\widetilde{AW}_k(s_j)$ is a sorted set that contains the top- k workers of task s_j ’s available worker set when $AW(s_j)$ is sorted descendingly according to the preferences of task s_j ’s available workers. Similarly, $\widetilde{RT}_k(w_i)$ is a set containing the top- k tasks of worker w_i ’s reachable task set when set $RT(w_i)$ is sorted descendingly according to the preference of worker w_i for task s , $\forall s \in RT(w_i)$. For each edge (v_i^w, v_j^s) , its weight, $weight(v_i^w, v_j^s)$, can be measured as the preference of worker w_i for task s_j , denoted as $p_{w_i}(s_j)$, which can be calculated as

$$p_{w_i}(s_j) = \exp \left(\langle h_{w_i}^z, e_{c(s_j)}^z \rangle \right), \quad (18)$$

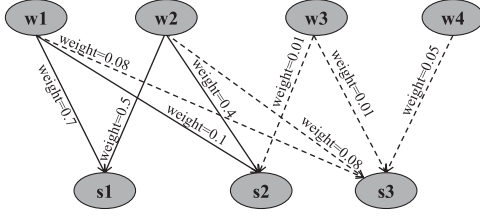


Fig. 5. Worker-task bipartite graph.

Algorithm 4: Intersected Top- k KM Algorithm.

Input: graph G , k , sorted available worker set $\widetilde{AW}$
Output: $match$

- 1 Initialize $match$, val_{task} and $slack$;
- 2 **for** each worker $w \in W$ **do**
- 3 $val_{worker}[w] \leftarrow \max(weight(v_w^W, v_s^S))$;
- 4 **for** each worker $w \in W$ **do**
- 5 **while** $val_{worker}[w] > 0$ **do**
- 6 Initialize vis_{task} and vis_{worker} to $False$;
- 7 **if** $FindTask(w, 0)$ **then** **break** ;
- 8 **else**
- 9 $d = INF$;
- 10 **for** each task $s \in S$ **do**
- 11 **if** $vis_{task}[s]$ **then**
- 12 $d \leftarrow \min(d, slack[s])$;
- 13 **for** each worker $w \in W$ **do**
- 14 Decrease $val_{worker}[w]$ by d if w is visited;
- 15 **for** each task $s \in S$ **do**
- 16 Increase $val_{task}[s]$ by d if s is visited;
- 17 Decrease $slack[s]$ by d if s is not visited;
- 18 $ReassignTask(match, \widetilde{AW})$;
- 19 **return** $match$;

where $h_{w_i}^z$ summarizes the category information of worker w_i 's recent task records, and $e_{c(s_j)}^z$ is the category embedding of category $c(s_j)$, which is the task s_j 's category.

Fig. 5 depicts a graph for four workers w_i ($i = 1, 2, 3, 4$) and three tasks s_j ($j = 1, 2, 3$). If vertices w_i and s_j satisfy both of the above constraints, edge (v_i^w, v_j^s) is drawn as a solid line (e.g., edge (v_1^w, v_1^s) when $k = 2$). If these two vertices only satisfy the spatio-temporal constraints, the edge (v_i^w, v_j^s) becomes a dotted line, which will be removed from the original graph in our algorithm. For example, edge (v_3^w, v_2^s) is drawn as a dotted line, because w_3 is not in the top-2 of $\widetilde{AW}_2(s_2)$. The higher $weight(v_i^w, v_j^s)$ of the edge (v_i^w, v_j^s) is, the more likely a worker is to perform this task successfully. Therefore, the mutual top- k will help filter the lower preference edge and keep the quality of assignments. However, this method will remove a number of edges and will contribute to a higher loss of the number of task assignments (e.g., w_4 and s_3). Therefore, we redistribute the task s_j to worker w_i if $w_i \in \widetilde{AW}(s_j)$ and w_i is not assigned to any task, where $\widetilde{AW}(s_j)$ is a sorted set that contains all available workers of task s_j .

The Intersected Top- k KM Algorithm is shown in Algorithm 4. Suppose that we have a bipartite graph G that is

Algorithm 5: FindTask Algorithm.

Input: worker w , recursion depth rdp
Output: $Bool$

- 1 $vis_{worker}[w] = True$;
- 2 **if** $rdp > \lambda$ **then** **return** $False$;
- 3 **else**
- 4 **for** each task s is adjacent to w in G **do**
- 5 **if** $vis_{task}[s]$ **then** **continue** ;
- 6 $gap \leftarrow$
- 7 $val_{worker}[w] + val_{task}[s] - weight(v_w^W, v_s^S)$;
- 8 **if** $gap \neq 0$ **then**
- 9 $slack[s] \leftarrow \min(slack[s], gap)$;
- 10 **else if** $match[s] == -1$ **or**
- 11 $FindTask(match[s], rdp+1)$ **then**
- 12 Assign value w to $match[s]$ and **return** $True$;
- 13 **return** $False$;

composed of two vertex sets V_S and V_W . First, the expectation of each vertex in V_W is equal to the largest weight among the edges associated with it (lines 2–3). Second, matching tasks for worker w are recursively found through the Find Task Algorithm 5 (line 7). Third, if w fails to match a task, to assign more tasks to workers, the expectations of workers and tasks involved in the last matching are adjusted, thus changing the competitive relationship among workers (lines 8–17). Some refinements should be made to the original KM algorithm, which is used to find the perfect matching of a weighted bipartite graph, to take into account cases where perfect matches do not exist [30]. Since the original KM algorithm may cause an endless loop in our problem, we stop matching tasks for the worker if the expectation of w is below 0 (line 5). The time complexity of the proposed Intersected Top- k KM Algorithm is $O(n \log N)$.

To improve efficiency, we limit the number of edges in the graph. Since we use the intersected Top- k method to filter edges, the number of edges associated with a worker node or a task node cannot exceed k . Thus, the recursion depth of Algorithm 5 is set to k , which can reduce the competition among workers (lines 2). The time complexity of the FindTask algorithm is $O(n)$. To maximize the number of task assignments, a reassignment task algorithm is designed—see Algorithm 6. First, the assignment result is checked (lines 1–3) in order to keep assignments satisfying the constraints proposed before. Then, we assign each of the remaining available tasks to the worker with the highest preference score among the corresponding available workers (lines 4–9). The time complexity of the reassignment task algorithm is $O(n)$.

IV. EXPERIMENTAL EVALUATION

A. Experimental Setup

We use a check-in dataset from Twitter, which is used widely for experimental evaluations of SC platforms [11], [31], [32], [33]. It provides check-in data in the United States from September 2010 to January 2011 and includes 62,462 venue locations and 61,412 user locations. First, we use FourSquare's

Algorithm 6: Reassignment Task Algorithm.

Input: KM output assignments $match$, $\widetilde{AW}$
Output: modified assignments $match$

```

1 for each task  $s$  in  $match$  do
2   if  $match[s] \neq -1$  and  $match[s] \notin \widetilde{AW}(s)$  then
3      $match[s] \leftarrow -1$ ;
4 for each task  $s$  in  $match$  do
5   if  $match[s] == -1$  and  $|\widetilde{AW}(s)| > 0$  then
6     for each worker  $w$  in  $\widetilde{AW}(s)$  do
7       if  $w$  not in  $match$  then
8         Assign value  $w$  to  $match[s]$ ;
9       return True;
10 return  $match$ ;

```

API¹ to generate the corresponding category information of venues. Then, for each worker and task, we take the average of the corresponding check-in locations as a venue's location information. For each check-in, we simulate that the user is a worker and that each venue accessed by the user is a task performed by the worker. The publication time of the task is set to the earliest check-in time of the task in a day. We use the category information of the venues in 18 kinds of check-ins to simulate the category information of tasks. Moreover, we randomly and uniformly generate 32 platform centers and use a Voronoi diagram-based algorithm [30] to allocate all workers and their historical data to corresponding platform centers. It is common practice in experimental studies of spatial crowdsourcing platforms to use uniformly distributed attribute values [34], [35], [36], the argument being that this captures the effects of the attributes on a more fair basis. A check-in record denotes that the worker has accepted and completed the task. All the experiments are implemented on an Intel(R) Xeon(R) CPU E5-2650 v4 @ 2.20 GHz and NVidia TITAN Xp GPU.

B. Performance of Federated Preference Learning

1) *Overall Accuracy:* We first evaluate the overall accuracy of the federated preference learning.

Evaluation Methods: We study four models. 1) CatePEP: The Category-based Personal Equal Popularity (CatePEP) model, where the popularity of task category c is set to 1 if the target worker did tasks in category c ; otherwise, it is set to 0. The popularity of task category c is considered as the worker's preference for c .

2) POISepPop: The POI-based Sequence Popularity (POISepPop) model, which ranks tasks according to their popularity in the target worker's sequence in a descending order. The popularity of task category c is calculated as $1/rank(c)$, which is regarded as the target worker's preference for c .

3) CTP: The Centralized Training based Preference (CTP) model, which trains our local preference model in a centralized manner. It means all workers belong to a single center.

TABLE II
OVERALL ACCURACY OF DIFFERENT PREFERENCE MODELS

Methods	Recall@1	Recall@2	Recall@3
CatePEP	0.1190	0.236	0.3630
POISepPop	0.3876	0.5210	0.6065
CTP	0.4204	0.5978	0.6994
FLP	<u>0.4120</u>	<u>0.5832</u>	<u>0.6850</u>

TABLE III
ACCURACY AND CONVERGENCE TIME OF DIFFERENT TRAINING MANNERS

Methods	Recall@1	Recall@2	Recall@3	Convergence Time
FLP	0.4120	0.5832	0.6850	17.97 min
FLP-FedAvg	0.4278	0.5927	0.6933	11.72 min
FLP-DCFT	<u>0.4267</u>	0.5946	0.6946	9.77 min

4) FLP: Our Federated Learning based Preference (FLP) model.

Metrics: To evaluate the accuracy of worker preference learning, we use $Recall@K$ as the evaluation metric. This metric counts the proportion of times when the categories of ground-truth tasks are ranked among the top- K predictions. We use workers' historical check-in sequences stored in local platform centers for the experiments and we use the first 80% of the check-in records for training, the next 10% for validation, and the remaining 10% for testing. We remove POIs associated with fewer than 5 records and workers with fewer than 10 or more than 300 check-in records.

Results: Table II shows the evaluation results, in which CTP performs best followed by FLP, POISepPop, and CatePEP. The models (CTP and FLP) based on our category-aware preference model outperform the others by far. This shows that our preference model can provide more accurate estimates of worker preferences. Considering that the local-device level empirical losses are inconsistent with the global empirical loss, it is expected that FLP performs worse than CTP. Moreover, with the aid of linear and quadratic penalty terms, the $Recall@K$ ($k = 1, 2, 3$) of FLP is 97.6%–98.0% of that of CTP.

2) *Effect of Different Federated Training Approaches:* We proceed to evaluate the performance of preference learning across different federated training approaches.

Evaluation Methods: We train the preference model according to three different approaches and report the performance.

- 1) FLP: Federated training (see Algorithm 1) is used for preference learning.
- 2) FLP-FedAvg: The preference learning is optimized using FedAvg (see Algorithm 2).
- 3) FLP-DCFT: The preference learning is optimized with drift-corrected federated training to overcome data heterogeneity (see Algorithm 3).

Metrics: We quantify the performance of preference learning with the different optimization approaches by measuring the accuracy (i.e., $Recall@k$) of worker preference learning and the convergence time. We set the ratio of sampled clients to 0.7 during the training of FLP-FedAvg and FLP-DCFT.

Results: In Table III, the overall best results are highlighted in bold. Fed-DCFT performs the best among all methods w.r.t.

¹<https://developer.foursquare.com/>

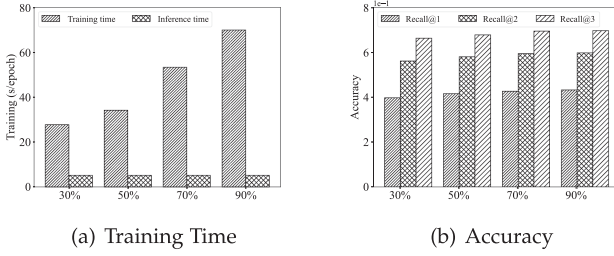


Fig. 6. Parameter sensitivity analysis.

Recall@2, and *Recall@3*: This is because DCT has the ability to contend with data heterogeneity by correcting client drift, which improves the performance of preference learning. As popular optimization methods, FLP-FedAvg and FLP achieve comparable results as both suffer from the issue of client drift, which affects the performance negatively.

To observe whether DCFT is able to accelerate the convergence of preference learning, we report the convergence time across different training approaches. We see that FLP-DCFT converges the fastest, followed by FLP-FedAvg and FLP. More specifically, FLP-DCFT converges faster than FedAvg by 16.64% and faster than FLP by 45.63%. This is because that FLP-DCFT only involves a subset of clients (i.e., 70%) for training and accelerates the training by performing drift correction, which makes it easier for the preference model to reach the optimum.

3) *Parameter Sensitivity of Preference Learning*: Next, we assess the sensitivity of FLP-DCFT in terms of the number of clients involved in the training and the number of hidden features in the local preference learning models.

Effect of the Number of Clients: We conduct experiments in which we randomly sample differently-sized subsets of clients (i.e., 30%, 50%, 70%, and 90% of clients) when training FLP-DCFT in each training round. Fig. 6(a) shows the training time across different ratios of clients. The training time increases from around 27 s to 70 s per epoch as the number of clients increases. This shows that involving more clients in the training can increase the training time, which is due to more training data being introduced. Next, we report the accuracy of FLP-DCFT in Fig. 6(b). We observe that *Recall@k* shows an increasing trend, meaning that involving more clients with more training data leads to better accuracy.

Effect of the Number of Hidden Features: We also study how sensitive the model is to the deep neural network structure with different parameters. As indicated in Fig. 7, we vary the number of channels (features) of the hidden layers in the local preference models from 16 to 256. We see that the model accuracy first increases and then decreases slightly with the increase of hidden features, indicating that the proposed preference model is robust to different parameters (i.e., hidden features) settings. We also observe that our local preference model achieves the best performance when setting the number of hidden features to 128. Therefore, we use 128 hidden features as the default in experiments.

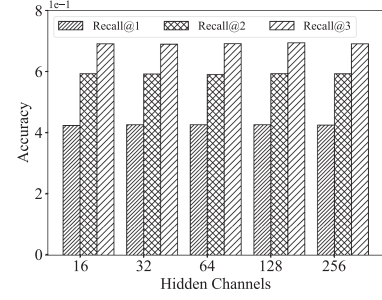


Fig. 7. Effect of hidden features.

TABLE IV
EXPERIMENT PARAMETERS

Parameter	Values
Valid time of tasks (h) $e - p$	0.4, 0.8, 1.2, 1.6, <u>2.0</u>
Reachable distance of workers (km) r	10, 15, 20, 25, <u>30</u>
Number of workers $ W $	2200, 2400, 2600, 2800, <u>3000</u>
Number of tasks $ S $	2600, 2800, 3000, 3200, <u>3400</u>
Limit coefficient k	10, 20, 30, 40, <u>50</u>

C. Performance of Task Assignment

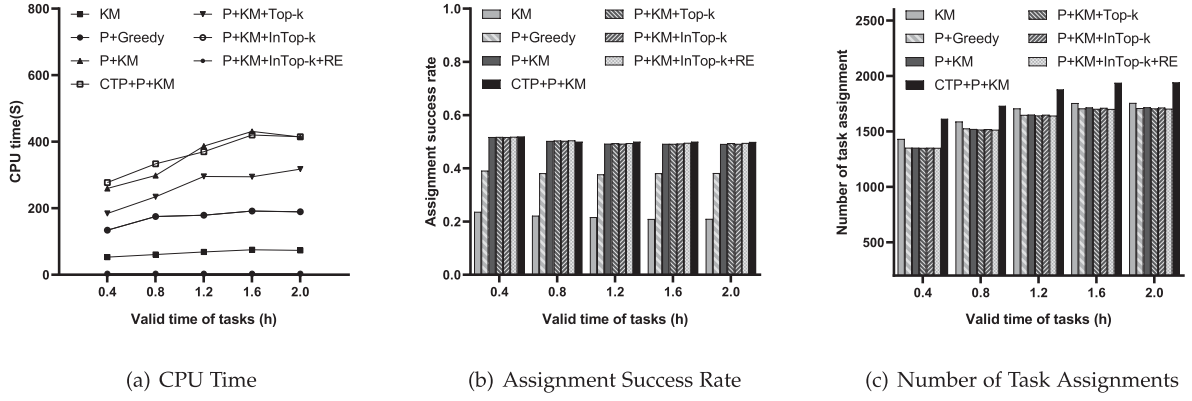
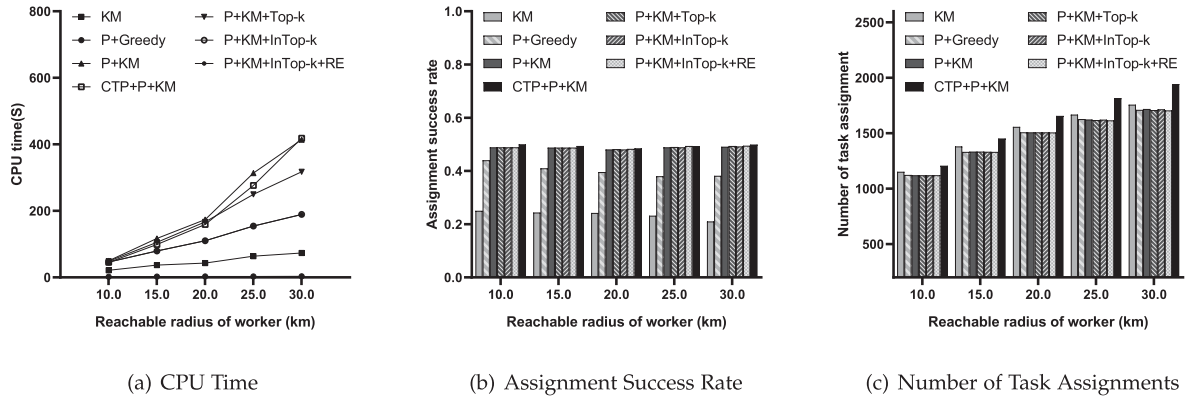
We proceed to study the performance of task assignment. Table IV shows our experimental settings, where the default values of all parameters are underlined.

Evaluation Methods: We study the following task assignment algorithms.

- 1) KM: The original KM algorithm that does not consider workers' preferences.
- 2) P+Greedy: The Greedy algorithm with workers' Preferences calculated by our FLP model.
- 3) P+KM: The original KM algorithm with workers' Preferences calculated by our FLP model.
- 4) P+KM+Top- k : The P+KM algorithm, using the worker-sided Top- k pruning strategy to prune edges.
- 5) P+KM+InTop- k : The P+KM algorithm, using our intersected Top- k pruning strategy to prune edges.
- 6) P+KM+InTop- k +RE: The P+KM algorithm, using our Intersected Top- k pruning strategy and the Reassignment optimization strategy.

7) CTP+P+KM: The original KM algorithm with workers' preferences calculated by CTP model. *Metrics*. Three metrics are compared among the methods, including CPU time, Assignment Success Rate (ASR), and the number of task assignments. The CPU time is the time cost for finding the task assignment. ASR is the ratio between the number of successful assignments of all workers and the total number of assignments in a time instance. In our experiments, if a worker performs (checks in) tasks (locations) with the same category in the next two check-ins, the assignment of this task can be considered successful.

Effect of $e - p$: First, we evaluate the effect of the valid time $e - p$ of tasks on the performance of task assignments (see Fig. 8). It can be seen from Fig. 8(a) and (c) that the CPU time and the number of task assignments of all algorithms exhibit an increasing trend as the valid time of tasks increases. This is because as the valid time of tasks increases, there will be

Fig. 8. Performance of task assignment: Effect of $e - p$.Fig. 9. Performance of task assignment: Effect of r .

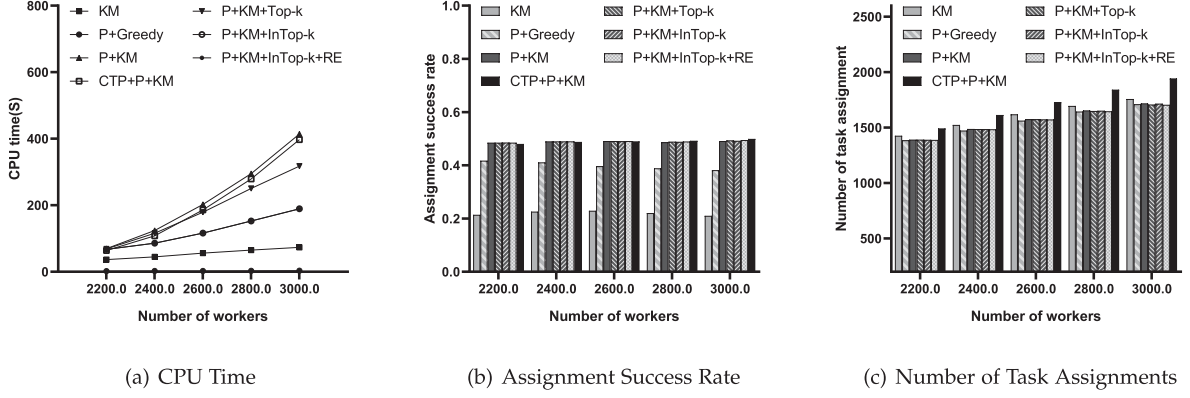
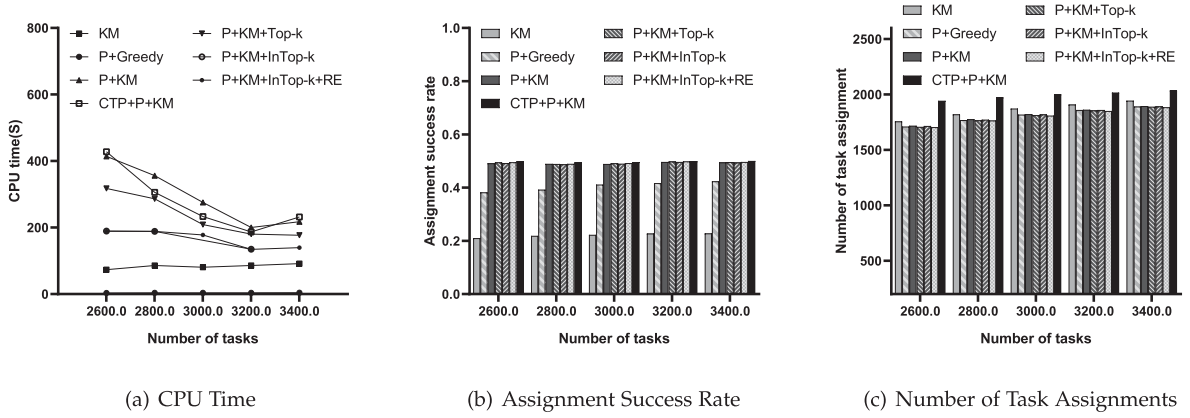
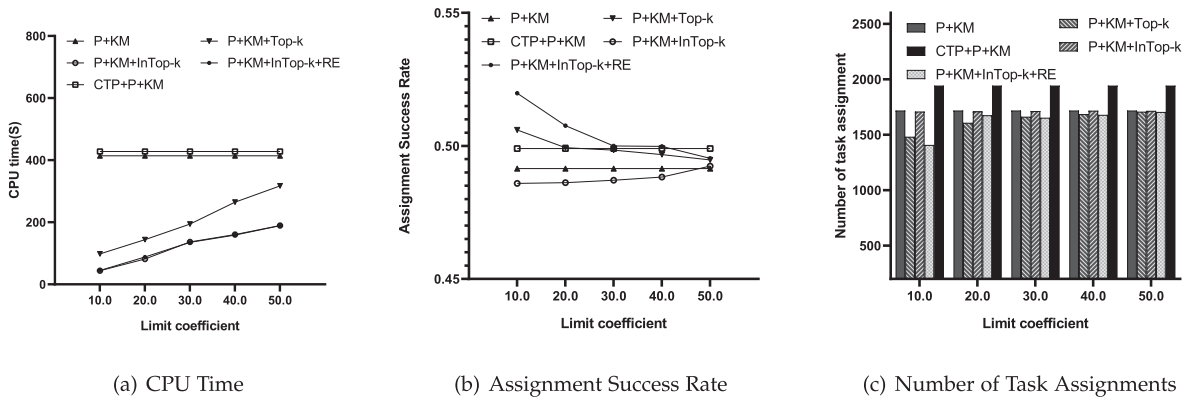
more available workers and tasks, which leads to a larger search space and higher probability of being assigned to a task for each worker. The CPU time of P+KM+InTop- k is less than those of P+KM+Top- k and P+KM while keeping almost the same ASR and number of task assignments, which is evidence of the efficiency and effectiveness of our proposed algorithms. As shown in Fig. 8(b), P+KM related algorithms achieve the highest Assignment Success Rate (ASR), which shows the importance of considering preferences. As shown in Fig. 8(c), the KM algorithm has the most task assignments, while other preference-related algorithms achieve fewer task assignments. This is due to the fact that the preferences of some workers among tasks vary greatly and many workers tend to choose the tasks they are interested in, thus leading to a lower number of task assignments. This reflects from the side that the preference scores we learn are accurate and discriminative.

Effect of r : We further evaluate the effect of the reachable distance r of workers. It can be seen from Fig. 9(a) that when r increases, the CPU time of all algorithms shows a similar growth trend. The reason is that when the reachable distance of workers increases, the number of available workers and the number of reachable tasks increase, yielding a larger search space. The P+Greedy algorithm still consumes the least CPU time, but its performance in ASR is obviously not as good as those of the other P+KM related algorithms (see Fig. 9(b)). In

addition, as shown in Fig. 9(c), when r increases, the number of task assignments also increases since workers are more likely to be assigned their available tasks with greater r .

Effect of $|W|$: Next, we evaluate the effect of $|W|$. As shown in Fig. 10(a), the larger $|W|$ is, the longer the CPU time is. This is because more available workers need to be assigned, which leads to more competition for limited tasks and generates more time overhead. When it comes to ASR in Fig. 10(b), all preference-based algorithms keep high ASR values, and the number of task assignments increases (cf. Fig. 10(c)). In summary, P+KM+InTop- k +RE, which considers privacy protection via federated learning, performs well in terms of CPU time and ASR while offering the acceptable number of task assignments. In addition, CTP+P+KM performs well than P+KM+InTop- k +RE. This is because CTP learns preference by centralized learning, however, which has high risks of privacy leakage.

Effect of $|S|$: We study the effect of the number $|S|$ of tasks. In Fig. 11(a), the CPU time of P+KM related algorithms decreases because as the number of tasks increases, the occurrence probability of tasks that workers are most interested in increases and the discriminative preference scores also disperse the competition among workers. Moreover, the CPU time of KM-related algorithms is higher than that of Greedy algorithm. Because it is time-consuming for KM-related algorithms to find a perfect matching with multiple iterations. The number of tasks assigned

Fig. 10. Performance of task assignment: Effect of $|W|$.Fig. 11. Performance of task assignment: Effect of $|S|$.Fig. 12. Performance of task assignment: Effect of k .

by KM is more than those of preference-based algorithms (i.e., all algorithms except KM), which sacrifice the number of task assignments in order to improve the total preferences of workers. In addition, with the increase of $|S|$, a worker can access more available and interested tasks with less competition, so both the number of task assignments and the ASR value increase. In terms of decentralized training, P+KM+InTop- k +RE performs better in terms of CPU time and ASR, showing its superiority.

Effect of k : Finally, we study the effect of k , which limits the number of edges associated with vertices and recursion depth in pruning-based algorithms (i.e., P+KM+Top- k , P+KM+InTop- k and P+KM+InTop- k +RE) and CTP+P+KM. We report the results of pruning-based algorithms, where P+KM is used as a reference. As k is a parameter only for the pruning-based algorithms, the results of P+KM in Fig. 12 keep the same value. In Fig. 12(a), InTop- k based algorithms (i.e., P+KM+InTop- k and

P+KM+InTop- k +RE) cost the least cpu time because they use the intersection operation to filter more edges with low weights. In the graphs of P+KM+Top- k and P+KM+InTop- k , each vertex is associated with more edges, leading to more opportunities to be assigned to available tasks as k increases. However, k has little impact on the high weight edge. Thus, the number of task assignments of these two algorithms increases while ASR decreases as k grows. Compared with P+KM+InTop- k , P+KM+InTop- k +RE takes less time but is more effective to keep the number of task assignments, which shows the necessity of the reassignment strategy. In addition, CTP+P+KM performs the best in task assignment due to the centralized training with more data.

V. RELATED WORK

Spatial crowdsourcing (SC) is a new framework that has emerged recently, requiring workers with GPS devices to reach a specific location physically under certain restrictions to perform spatial tasks [12], [15], [30], [37], [38], [39], [40], [41], [42], [43], [44], [45], [46]. Most of the existing studies focus on task assignment [25], [47], [48], [49], [50], [51], [52], [53], however, putting their focus on effectiveness without considering the privacy of users' raw data and the tediousness of data migration in reality, which leads to the risk of privacy leaks. To make the SC server assign tasks properly, workers need to upload their highly sensitive data (e.g., locations and historical task records), which disclose their private attributes. Thus, in recent studies [5], [54], privacy-preserving task assignment is proposed to make users (workers/task requesters) perturb their locations with Geo-Indistinguishability [16] and upload only the perturbed locations. However, these studies mainly focus on the privacy of locations without considering the preferences of workers, failing to achieve a satisfying task assignment. With federated learning, multiple entities (clients) collaborate to solve a problem, under the coordination of a central server or service provider [20]. Each client's raw data is stored locally and not exchanged or transferred; instead, focused updates intended for immediate aggregation are used to achieve the learning objective [21], [23], [29], [55]. For example, a novel quantized federated averaging algorithm [55] is designed to apply a stochastic quantization scheme to the local and global model parameters, which is the first work to provide theoretical analysis for quantized federated learning algorithms with convex functions. In this paper, we propose a federated preference learning model to protect the privacy of workers' raw data and learn worker preferences without data migration, based on which we assign tasks to suitable workers.

VI. CONCLUSION

We propose a framework called Task Assignment with Federated Preference Learning (TA-FPL), which aims to find optimal task assignments while considering workers' preferences and protecting workers' raw data. TA-FPL consists of an efficient Federated Preference Learning (FPL) phase and a Preference-driven Task Assignment (PTA) phase. For the FPL phase, we present local means of learning platform center models and combine these with a drift-corrected federated training method.

For the PTA phase, we propose an Intersected Top- k KM algorithm to achieve effective and efficient task assignments based on worker preferences obtained in the first phase. To the best of our knowledge, this is the first study in SC that applies federated learning and enables preference modeling while offering raw data privacy. An empirical study based on a real dataset offers evidence of the superiority of our proposed algorithms. In future research, it is of interest to achieve privacy-preserving clustering so that similar clients can be clustered, thereby further accelerating preference learning.

REFERENCES

- [1] Y. Tong, Z. Zhou, Y. Zeng, L. Chen, and C. Shahabi, "Spatial crowdsourcing: A survey," *VLDB J.*, vol. 29, no. 1, pp. 217–250, 2020.
- [2] S. R. B. Gummid, X. Xie, and T. B. Pedersen, "A survey of spatial crowdsourcing," *ACM Trans. Database Syst.*, vol. 44, no. 2, pp. 1–46, 2019.
- [3] G. Li, Y. Zheng, J. Fan, J. Wang, and R. Cheng, "Crowdsourced data management: Overview and challenges," in *Proc. ACM Int. Conf. Manage. Data*, 2017, pp. 1711–1716.
- [4] H. To, G. Ghinita, and C. Shahabi, "A framework for protecting worker location privacy in spatial crowdsourcing," in *Proc. VLDB Endowment*, vol. 7, no. 10, pp. 919–930, 2014.
- [5] H. To, C. Shahabi, and L. Xiong, "Privacy-preserving online task assignment in spatial crowdsourcing with untrusted server," in *Proc. IEEE 34th Int. Conf. Data Eng.*, 2018, pp. 833–844.
- [6] X. Yi, F.-Y. Rao, G. Ghinita, and E. Bertino, "Privacy-preserving spatial crowdsourcing based on anonymous credentials," in *Proc. 19th IEEE Int. Conf. Mobile Data Manage.*, 2018, pp. 187–196.
- [7] C. Qiu and A. C. Squicciarini, "Location privacy protection in vehicle-based spatial crowdsourcing via geo-indistinguishability," in *Proc. IEEE 39th Int. Conf. Distrib. Comput. Syst.*, 2019, pp. 1061–1071.
- [8] C. Qiu, A. Squicciarini, Z. Li, C. Pang, and L. Yan, "Time-efficient geobfuscation to protect worker location privacy over road networks in spatial crowdsourcing," in *Proc. 29th ACM Int. Conf. Inf. Knowl. Manage.*, 2020, pp. 1275–1284.
- [9] M. Li et al., "Privacy-preserving batch-based task assignment in spatial crowdsourcing with untrusted server," in *Proc. 30th ACM Int. Conf. Inf. Knowl. Manage.*, 2021, pp. 947–956.
- [10] W. Huang, X. Lei, and H. Huang, "PTA-SC: Privacy-preserving task allocation for spatial crowdsourcing," in *Proc. IEEE Wireless Commun. Netw. Conf.*, 2021, pp. 1–7.
- [11] Y. Li, Y. Zhao, and K. Zheng, "Preference-aware group task assignment in spatial crowdsourcing: A mutual information-based approach," in *Proc. IEEE Int. Conf. Data Mining*, 2021, pp. 350–359.
- [12] Z. Wang, Y. Zhao, X. Chen, and K. Zheng, "Task assignment with worker churn prediction in spatial crowdsourcing," in *Proc. 30th ACM Int. Conf. Inf. Knowl. Manage.*, 2021, pp. 2070–2079.
- [13] X. Li, Y. Zhao, J. Guo, and K. Zheng, "Group task assignment with social impact-based preference in spatial crowdsourcing," in *Proc. Int. Conf. Database Syst. Adv. Appl.*, 2020, pp. 677–693.
- [14] Y. Zhao, K. Zheng, H. Yin, G. Liu, J. Fang, and X. Zhou, "Preference-aware task assignment in spatial crowdsourcing: From individuals to groups," *IEEE Trans. Knowl. Data Eng.*, vol. 34, no. 7, pp. 3461–3477, Jul. 2022.
- [15] Y. Zhao et al., "Preference-aware task assignment in spatial crowdsourcing," in *Proc. 33rd AAAI Conf. Artif. Intell.*, 2019, pp. 2629–2636.
- [16] M. E. Andrés, N. E. Bordenabe, K. Chatzikokolakis, and C. Palamidessi, "Geo-indistinguishability: Differential privacy for location-based systems," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, 2013, pp. 901–914.
- [17] Q. Tao, Y. Tong, S. Li, Y. Zeng, Z. Zhou, and K. Xu, "A differentially private task planning framework for spatial crowdsourcing," in *Proc. 22nd IEEE Int. Conf. Mobile Data Manage.*, 2021, pp. 9–18.
- [18] Y. Wang et al., "Fed-LTD: Towards cross-platform ride hailing via federated learning to dispatch," in *Proc. 28th ACM SIGKDD Conf. Knowl. Discov. Data Mining*, 2022, pp. 4079–4089.
- [19] Y. Tong et al., "Hu-Fu: Efficient and secure spatial queries over data federation," in *Proc. VLDB Endowment*, vol. 15, no. 6, pp. 1159–1172, 2022.
- [20] P. Kairouz et al., "Advances and open problems in federated learning," *Found. Trends Mach. Learn.*, vol. 14, no. 1/2, pp. 1–210, 2021.

- [21] B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. y Arcas, "Communication-efficient learning of deep networks from decentralized data," in *Proc. 20th Int. Conf. Artif. Intell. Statist.*, 2017, pp. 1273–1282.
- [22] J. Liu, L. Deng, H. Miao, Y. Zhao, and K. Zheng, "Task assignment with federated preference learning in spatial crowdsourcing," in *Proc. 31st ACM Int. Conf. Inf. Knowl. Manage.*, 2022, pp. 1279–1288.
- [23] T. Li, A. K. Sahu, M. Zaheer, M. Sanjabi, A. Talwalkar, and V. Smith, "Federated optimization in heterogeneous networks," in *Proc. Mach. Learn. Syst.*, vol. 2, pp. 429–450, 2020.
- [24] S. P. Karimireddy, S. Kale, M. Mohri, S. J. Reddi, S. U. Stich, and A. T. Suresh, "SCAFFOLD: Stochastic controlled averaging for on-device federated learning," 2019, *arXiv:1910.06378*.
- [25] L. Kazemi and C. Shahabi, "GeoCrowd: Enabling query answering with spatial crowdsourcing," in *Proc. 20th Int. Conf. Adv. Geographic Inf. Syst.*, 2012, pp. 189–198.
- [26] A. Vaswani et al., "Attention is all you need," in *Proc. 31st Int. Conf. Neural Inf. Process. Syst.*, 2017, pp. 5998–6008.
- [27] J. Li, Y. Wang, and J. McAuley, "Time interval aware self-attention for sequential recommendation," in *Proc. 13th Int. Conf. Web Search Data Mining*, 2020, pp. 322–330.
- [28] R. Cai, J. Wu, A. San, C. Wang, and H. Wang, "Category-aware collaborative sequential recommendation," in *Proc. 44th Int. ACM SIGIR Conf. Res. Develop. Inf. Retrieval*, 2021, pp. 388–397.
- [29] A. E. Durmus, Z. Yue, M. Ramon, M. Matthew, W. Paul, and S. Venkatesh, "Federated learning based on dynamic regularization," in *Proc. Int. Conf. Learn. Representations*, 2021, pp. 1–36. [Online]. Available: <https://openreview.net/forum?id=B7v4QMR6Z9w>
- [30] G. Ye, Y. Zhao, X. Chen, and K. Zheng, "Task allocation with geographic partition in spatial crowdsourcing," in *Proc. 30th ACM Int. Conf. Inf. Knowl. Manage.*, 2021, pp. 2404–2413.
- [31] P. Cheng, X. Lian, L. Chen, and C. Shahabi, "Prediction-based task assignment in spatial crowdsourcing," in *Proc. IEEE 33rd Int. Conf. Data Eng.*, 2017, pp. 997–1008.
- [32] H. Dang, T. Nguyen, and H. To, "Maximum complex task assignment: Towards tasks correlation in spatial crowdsourcing," in *Proc. Int. Conf. Inf. Integr. Web-Based Appl. Serv.*, 2013, pp. 77–81.
- [33] D. Deng, C. Shahabi, and U. Demiryurek, "Maximizing the number of worker's self-selected tasks in spatial crowdsourcing," in *Proc. 21st Int. Conf. Adv. Geographic Inf. Syst.*, 2013, pp. 324–333.
- [34] P. Cheng, X. Jian, and L. Chen, "An experimental evaluation of task assignment in spatial crowdsourcing," in *Proc. VLDB Endowment*, vol. 11, no. 11, pp. 1428–1440, 2018.
- [35] Y. Tong, J. She, B. Ding, L. Wang, and L. Chen, "Online mobile micro-task allocation in spatial crowdsourcing," in *Proc. IEEE 32nd Int. Conf. Data Eng.*, 2016, pp. 49–60.
- [36] S. R. B. Gummid, T. B. Pedersen, and X. Xie, "Transit-based task assignment in spatial crowdsourcing," in *Proc. 32nd Int. Conf. Sci. Statist. Database Manage.*, 2020, pp. 1–12.
- [37] Y. Cui, L. Deng, Y. Zhao, B. Yao, V. W. Zheng, and K. Zheng, "Hidden POI ranking with spatial crowdsourcing," in *Proc. 25th ACM SIGKDD Conf. Knowl. Discov. Data Mining*, 2019, pp. 814–824.
- [38] Y. Tong, J. She, B. Ding, L. Chen, T. Wo, and K. Xu, "Online minimum matching in real-time spatial data: Experiments and analysis," in *Proc. VLDB Endowment*, vol. 9, no. 12, pp. 1053–1064, 2016.
- [39] Y. Tong, Y. Zeng, Z. Zhou, L. Chen, J. Ye, and K. Xu, "A unified approach to route planning for shared mobility," in *Proc. VLDB Endowment*, vol. 11, no. 11, pp. 1633–1646, 2018.
- [40] J. Tu, P. Cheng, and L. Chen, "Quality-assured synchronized task assignment in crowdsourcing," *IEEE Trans. Knowl. Data Eng.*, vol. 33, no. 3, pp. 1156–1168, Mar. 2021.
- [41] Y. Zhao, K. Zheng, Y. Cui, H. Su, F. Zhu, and X. Zhou, "Predictive task assignment in spatial crowdsourcing: A data-driven approach," in *Proc. IEEE 36th Int. Conf. Data Eng.*, 2020, pp. 13–24.
- [42] Y. Zhao, K. Zheng, J. Guo, B. Yang, T. B. Pedersen, and C. S. Jensen, "Fairness-aware task assignment in spatial crowdsourcing: Game-theoretic approaches," in *Proc. IEEE 37th Int. Conf. Data Eng.*, 2021, pp. 265–276.
- [43] Y. Zhao et al., "Outlier detection for streaming task assignment in crowdsourcing," in *Proc. ACM Web Conf.*, 2022, pp. 1933–1943.
- [44] Y. Zhao, K. Zheng, Y. Li, H. Su, J. Liu, and X. Zhou, "Destination-aware task assignment in spatial crowdsourcing: A worker decomposition approach," *IEEE Trans. Knowl. Data Eng.*, vol. 32, no. 12, pp. 2336–2350, Dec. 2020.
- [45] X. Li, Y. Zhao, X. Zhou, and K. Zheng, "Consensus-based group task assignment with social impact in spatial crowdsourcing," *Data Sci. Eng.*, vol. 5, no. 4, pp. 375–390, 2020.
- [46] Y. Zhao, J. Guo, X. Chen, J. Hao, X. Zhou, and K. Zheng, "Coalition-based task assignment in spatial crowdsourcing," in *Proc. IEEE 37th Int. Conf. Data Eng.*, 2021, pp. 241–252.
- [47] L. Kazemi, C. Shahabi, and L. Chen, "GeoTruCrowd: Trustworthy query answering with spatial crowdsourcing," in *Proc. 21st Int. Conf. Adv. Geographic Inf. Syst.*, 2013, pp. 314–323.
- [48] H. To, C. Shahabi, and L. Kazemi, "A server-assigned spatial crowdsourcing framework," *ACM Trans. Spatial Algorithms Syst.*, vol. 1, no. 1, pp. 1–28, 2015.
- [49] Y. Tong, L. Chen, Z. Zhou, H. V. Jagadish, L. Shou, and W. Lv, "SLADE: A smart large-scale task decomposer in crowdsourcing," *IEEE Trans. Knowl. Data Eng.*, vol. 30, no. 8, pp. 1588–1601, Aug. 2018.
- [50] Y. Tong et al., "Flexible online task assignment in real-time spatial data," in *Proc. VLDB Endowment*, vol. 10, no. 11, pp. 1334–1345, 2017.
- [51] P. Cheng, X. Lian, L. Chen, J. Han, and J. Zhao, "Task assignment on multi-skill oriented spatial crowdsourcing," *IEEE Trans. Knowl. Data Eng.*, vol. 28, no. 8, pp. 2201–2215, Aug. 2016.
- [52] J. Xia, Y. Zhao, G. Liu, J. Xu, M. Zhang, and K. Zheng, "Profit-driven task assignment in spatial crowdsourcing," in *Proc. 28th Int. Joint Conf. Artif. Intell.*, 2019, pp. 1914–1920.
- [53] L. Zheng, L. Chen, and J. Ye, "Order dispatch in price-aware ridesharing," in *Proc. VLDB Endowment*, vol. 11, no. 8, pp. 853–865, 2018.
- [54] Q. Tao, Y. Tong, Z. Zhou, Y. Shi, L. Chen, and K. Xu, "Differentially private online task assignment in spatial crowdsourcing: A tree-based approach," in *Proc. IEEE 36th Int. Conf. Data Eng.*, 2020, pp. 517–528.
- [55] Y. Li, W. Li, and Z. Xue, "Federated learning with stochastic quantization," *Int. J. Intell. Syst.*, vol. 37, no. 12, pp. 11600–11621, 2022.



Hao Miao received the ME degree in computer science and technology from the Nanjing University of Aeronautics and Astronautics, Nanjing, China, in 2021. He is currently working toward the PhD degree in computer science with Aalborg University, Denmark. From September 2019 to November 2019, he was a visiting student with PolyU, Hong Kong, China. His research interests include spatio-temporal data analytics, incremental learning, and federated learning.



Xiaolong Zhong received the bachelor's degree from the Chongqing University of Technology, in 2018. He is currently working toward the master's degree in computer science and technology with the University of Electronic Science and Technology of China. His research interests include federated learning and spatio-temporal data mining.



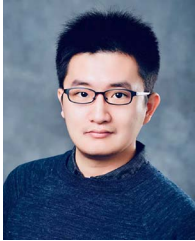
Jiaxin Liu received the bachelor's degree in computer science and technology from Southwest Jiaotong University, in 2021. She is currently working toward the master's degree in computer science and technology with the University of Electronic Science and Technology of China. Her research interests include spatial crowdsourcing, trajectory data mining and spatio-temporal databases.



Yan Zhao received the doctoral degree in computer science from Soochow University, China, in 2020. She is an associate professor with Aalborg University, Denmark. Her research interests include spatial databases and trajectory computing.



Weizhu Qian received the PhD degree in computer science from Université Bourgogne Franche-Comté, in 2021. He is a lecturer with Soochow University, China. He was a postdoctoral researcher with Aalborg University. His research interests mainly focus on deep learning and data science.



Xiangyu Zhao received the PhD degree from MSU, in 2021. He is an Assistant Professor of the School of Data Science, City University of Hong Kong (CityU). His current research interests include data mining and machine learning, especially their applications in Urban Computing and Recommender Systems. He has published more than 80 papers in top conferences and journals. His research has been awarded ICDM'22 and ICDM'21 Best-ranked Papers, Global Top 100 Chinese New Stars in AI, CCF-Ant Research Fund, CCF-Tencent Open Fund, Criteo Faculty Research

Award, and Bytedance Research Collaboration Award. He serves as top data science conference (senior) program committee members and session chairs (e.g., KDD, WWW, SIGIR, IJCAI, AAAI, ICML, ICLR), and journal guest editors and reviewers (e.g., *IEEE Transactions on Knowledge and Data Engineering*, *ACM Transactions on Knowledge Discovery from Data*, *ACM Transactions on Information Systems*, *ACM Transactions on Intelligent Systems and Technology*, *ACM Computing Surveys*, *Frontiers in Big Data*). Please find more information at <https://zhaoxyai.github.io/>.



Kai Zheng (Senior Member, IEEE) received the PhD degree in computer science from The University of Queensland, in 2012. He is a professor of computer science with the University of Electronic Science and Technology of China. He has been working in the area of spatial-temporal databases, uncertain databases, social-media analysis, in-memory computing and blockchain technologies. He has published more than 100 papers in prestigious journals and conferences in data management field such as SIGMOD, ICDE, *VLDB Journal*, *ACM Transactions* and *IEEE Transactions*.



Christian S. Jensen (Fellow, IEEE) is a professor of computer science with Aalborg University, Denmark. He was a professor with Aarhus University from 2010 to 2013, and he was previously with Aalborg University for two decades. His research concerns data management and analytics, and its focus is on temporal and spatio-temporal data. He is a member of the Academia Europaea, the Royal Danish Academy of Sciences and Letters, and the Danish Academy of Technical Sciences. He is a fellow of the ACM.